

İĞDIR ÜNİVERSİTESİ
YAZILIM MÜHENDİSLİĞİ

Dersin Adı: Grafik Programlama

Dersi Veren Öğretim Üyesi: Arş. Gör. Dr. Mustafa KORKMAZ

ÖDEV RAPORU

Belge Başlığı: 1. Ödev - Parametrik Grafik Çizici Uygulaması

Hazırlayan: Polat Bilir

Öğrenci Numarası: 2301060020

Teslim Tarihi: 6 Nisan 2026

İçindekiler

Amaç.....	3
Kullanılan Denklemlerin Açıklaması.....	3
Program Yapısı ve Akış Şeması.....	4
Ekran Görüntüleri.....	5-9
Karşılaşılan Zorluklar ve Çözüm Yolları.....	10
Kaynakça.....	11

1. Amaç

Bu projenin temel amacı, bilgisayar grafiklerinde karmaşık geometrilerin modellenmesinde yaygın olarak kullanılan parametrik denklemlerin, Python programlama dili ve Pygame kütüphanesi aracılığıyla interaktif bir yazılıma dönüştürülmesidir.

Bilgisayar grafiğinde eğriler üç farklı şekilde (Explicit, Implicit ve Parametric) ifade edilebilir. Explicit ($y = f(x)$) ve Implicit ($F(x,y) = 0$) formlar sınırlı ve çizimi zor yöntemlerken; x ve y koordinatlarının bağımsız bir 't' zaman/açı parametresine bağlandığı Parametrik form, kapalı döngülerin ve çok-değerli fonksiyonların modellenmesinde en esnek yöntemdir. Geliştirilen bu uygulama ile kullanıcıdan alınan dinamik parametreler (yarıçap, dönüş sayısı, kenar sayısı) anlık olarak işlenerek noktasal veri setlerine dönüştürülmüş ve bilgisayar ekranında piksellere dökülmüştür.

2. Kullanılan Denklemlerin Matematiksel Açıklaması

Uygulama kapsamında geliştirilen 10 farklı geometrik şekil, Kartezyen ve Polar (Kutupsal) koordinat sistemlerinin bilgisayar ortamına uyarlanmasıyla elde edilmiştir. Sürekli (continuous) olan matematiksel fonksiyonlar, bilgisayarın çizebilmesi için 't' radyan açısının belirli adım aralıklarıyla ($\Delta t = 0.05$) artırılmasıyla ayrık (discrete) veri noktalarına dönüştürülmüştür.

Temel formüller ve teorik karşılıkları aşağıda sunulmuştur:

- Daire ve Elips (Merkezi Simetri): Kutupsal sistemin en temel kullanım alanıdır. Dairede sabit bir yarıçap (r) kullanılırken, elipste yatay (a) ve dikey (b) yarıçap eksenleri ayrıştırılmıştır.

$$\text{Daire: } x = r * \cos(t) \mid y = r * \sin(t)$$

$$\text{Elips: } x = a * \cos(t) \mid y = b * \sin(t)$$

- Archimedean Spiral (Arşimet Spirali): Merkeze olan uzaklığın dönme açısıyla doğru orantılı olarak arttığı bir yapıdadır. Başlangıç mesafesi (a) ve dönüş başına artış katsayısı (b) kullanılarak her bir t anındaki konum hesaplanır

$$\text{Formül: } x = (a + bt) * \cos(t) \mid y = (a + bt) * \sin(t)$$

- **Kardioid ve Rozet (Polar Çiçek Eğrileri):** Merkezi simetriye sahip bu şekiller, aslında $r = f(\theta)$ şeklindeki basit kutupsal formüllerin kartezyene yansıtılmasıdır. Rozet eğrisinde n sayısı yaprak miktarını belirlerken, Kardioid eğrisi bir çemberin etrafında dönen başka bir çemberin izlediği yolu simüle eder.
 - Kardioid: $x = a * (2\cos(t) - \cos(2t))$ | $y = a * (2\sin(t) - \sin(2t))$
 - Rozet: $r = a * \cos(n*t)$ eşitliğinden yola çıkılarak; $x = r * \cos(t)$ | $y = r * \sin(t)$
- **Çokgensel Şekiller (Kare, Dikdörtgen, Eşkenar Üçgen, Yıldız):** Bu şekiller t parametresinin sürekli akışından ziyade, köşelerin kartezyen düzlemde belirli açılarla $(2\pi/n)$ dağıtılması mantığıyla explicit olarak hesaplanmıştır. Düzgün çokgende köşeler çember üzerine eşit açılarla paylaştırılırken, yıldız çiziminde iç (r) ve dış (R) çemberler arasında zikzak yapan ardışık bir döngü kurulmuştur.
 - formül= $R * \cos(2\pi k/n - \pi/2)$ ve $y = R * \sin(2\pi k/n - \pi/2)$

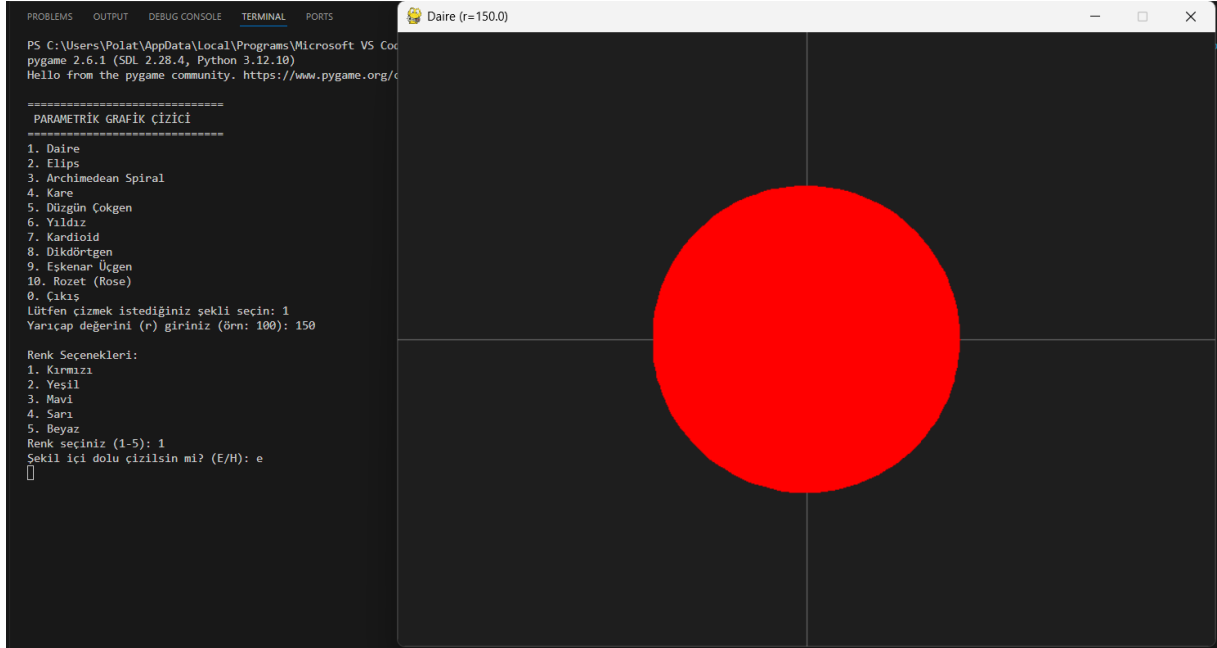
3. Program Yapısı ve Akış Şeması

Proje; Kullanıcı Arayüzü (Konsol), Hesaplama Motoru (Matematiksel Modüller) ve Görüntü İşleyici (Pygame Rendering) olmak üzere üç katmanlı bir mimaride (Separation of Concerns prensibine uygun olarak) tasarlanmıştır.

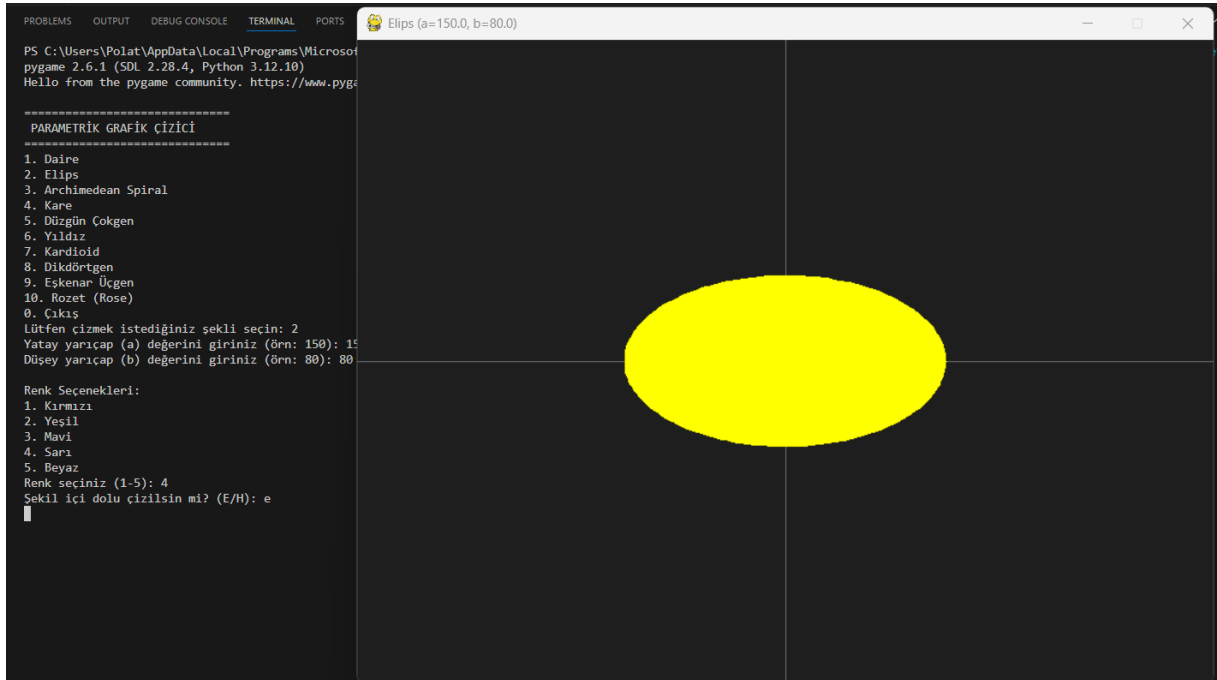
Akış Şeması (Adım Adım İzge):

- **Veri Toplama:** Ana menü (while) döngüsü başlar. Kullanıcıdan şekil tipi, renk kodu (RGB), boyut değişkenleri ve dolgu (polygon/lines) tercihleri "try-except" hata denetim bloklarından geçirilerek alınır.
- **Matematiksel Hesaplama:** Seçilen şekle ait modül çağrılır. Başlangıç noktası $t=0$ 'dan başlanarak maksimum t sınırına kadar döngü çalıştırılır. Her adımda ham (x, y) verileri üretilir.
- **Koordinat Dönüşümü (Mapping):** Üretilen ham matematiksel veriler $donusum(x,y)$ fonksiyonuna gönderilir. Ekran çözünürlüğünün merkezi (400, 300) referans alınarak $Ekran_X = Merkez_X + x$ ve $Ekran_Y = Merkez_Y - y$ formülü uygulanır.
- **Render (Çizim):** Elde edilen son ekran koordinat dizisi Pygame motoruna aktarılır. Kullanıcının tercihi göre şekil ya çizgisel bir yol (path) olarak ya da içi dolu çokgen (polygon) olarak ekrana yansıtılır.
- **Döngü Yönetimi:** Event listener (olay dinleyici) pencere kapanma tetikleyicisini bekler. Kapanma durumunda bellek temizlenerek kullanıcı yeniden ana menüye yönlendirilir.

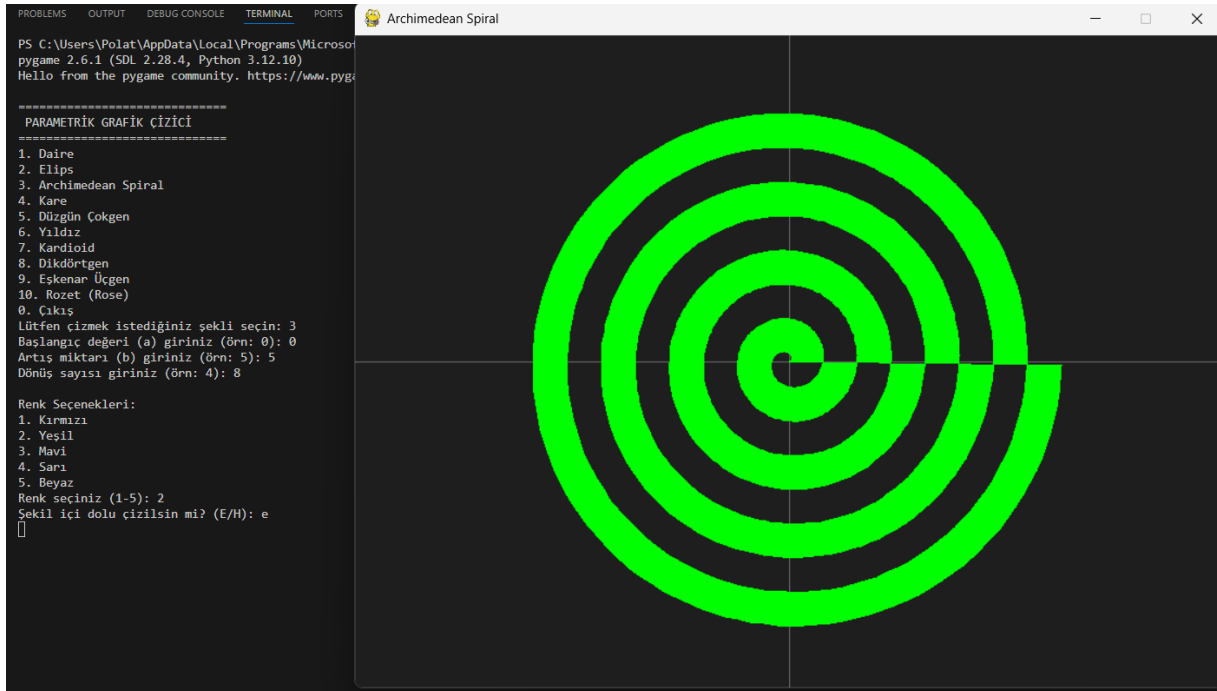
4. Ekran Görüntüleri ve Görsel Analiz



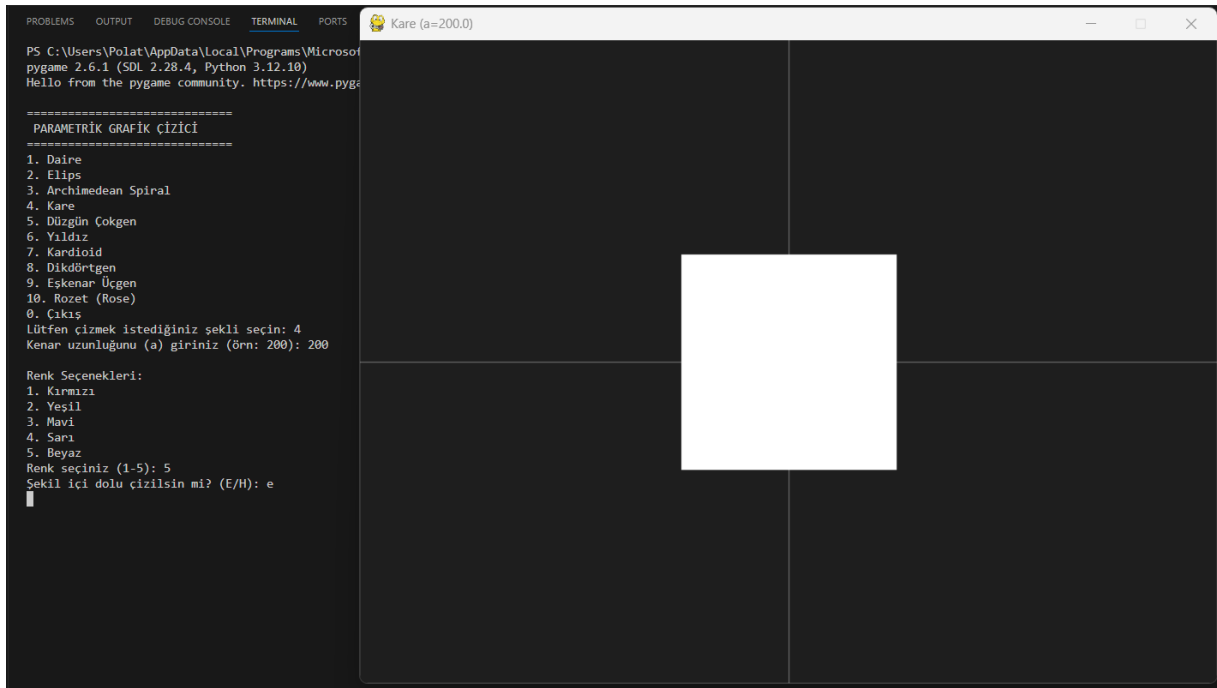
Şekil 1: Kırmızı renkli, içi dolu daire çizimi



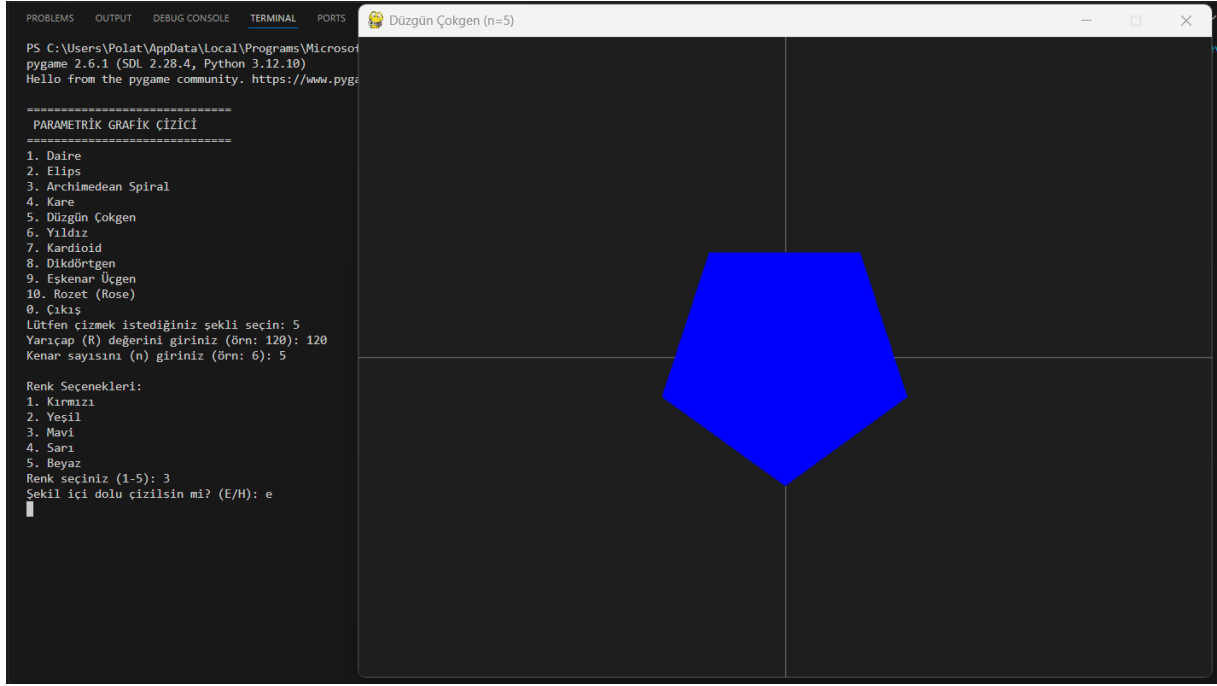
Şekil 2: Sarı renkli, içi dolu Elips çizimi



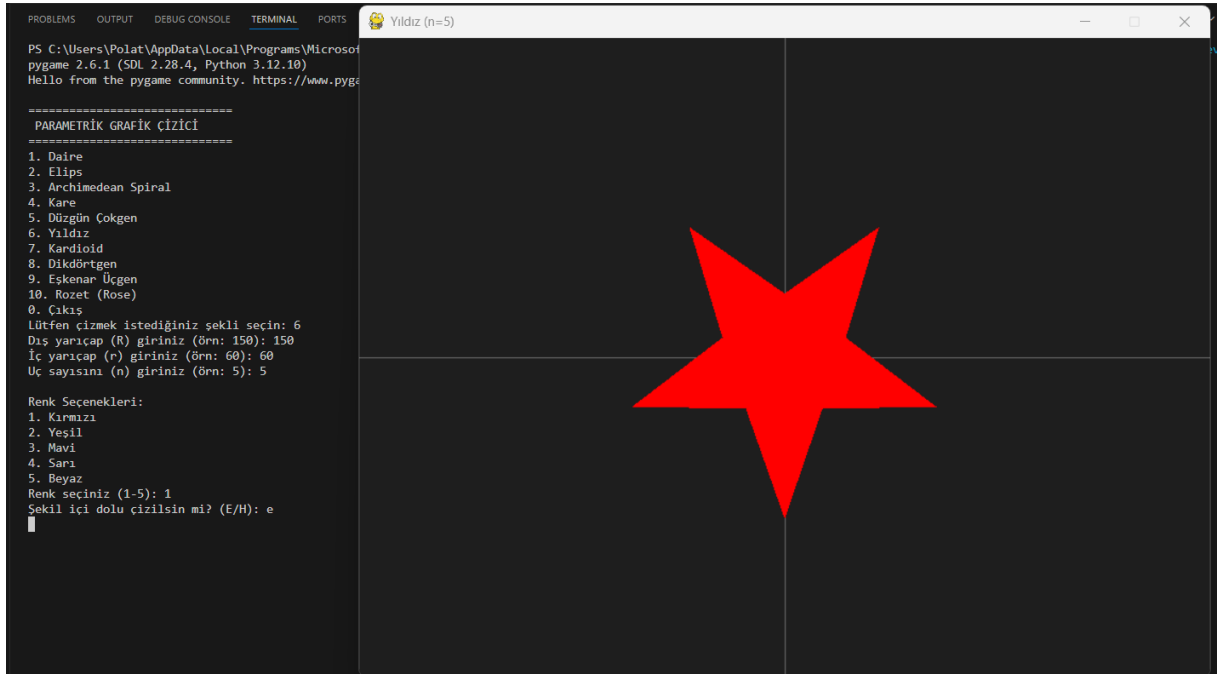
Şekil 3: Yeşil renkli, içi dolu Archimedean Spiral çizimi



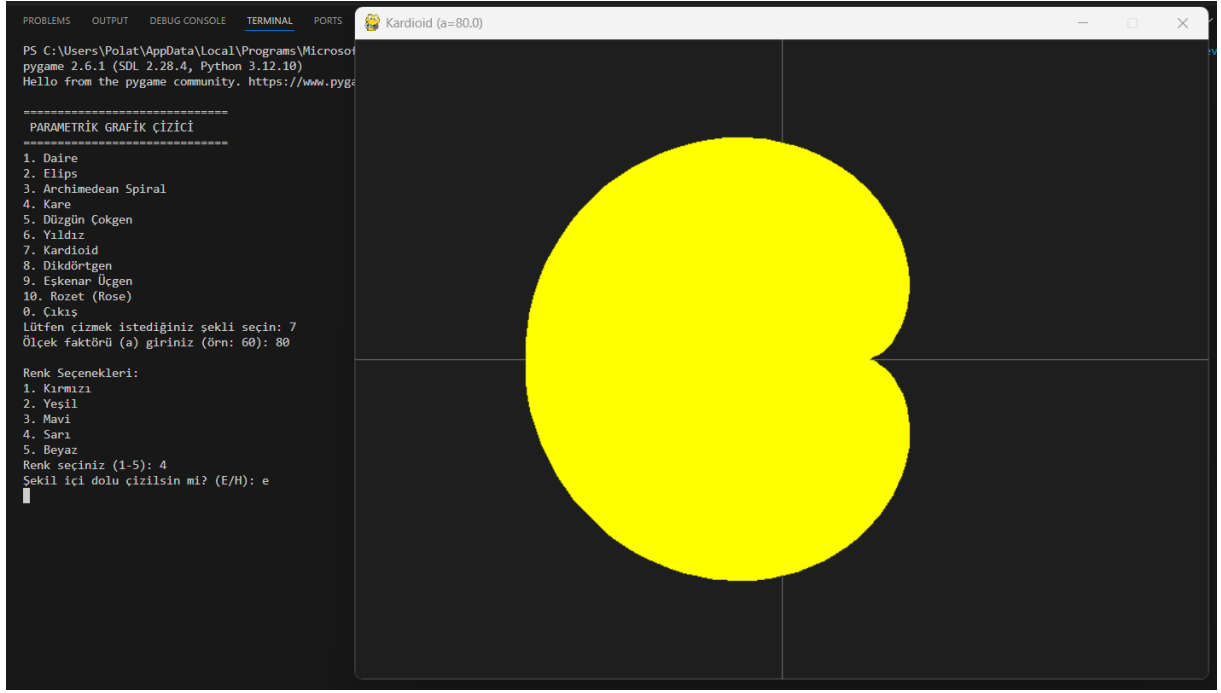
Şekil 4: Beyaz renkli, içi dolu Kare çizimi



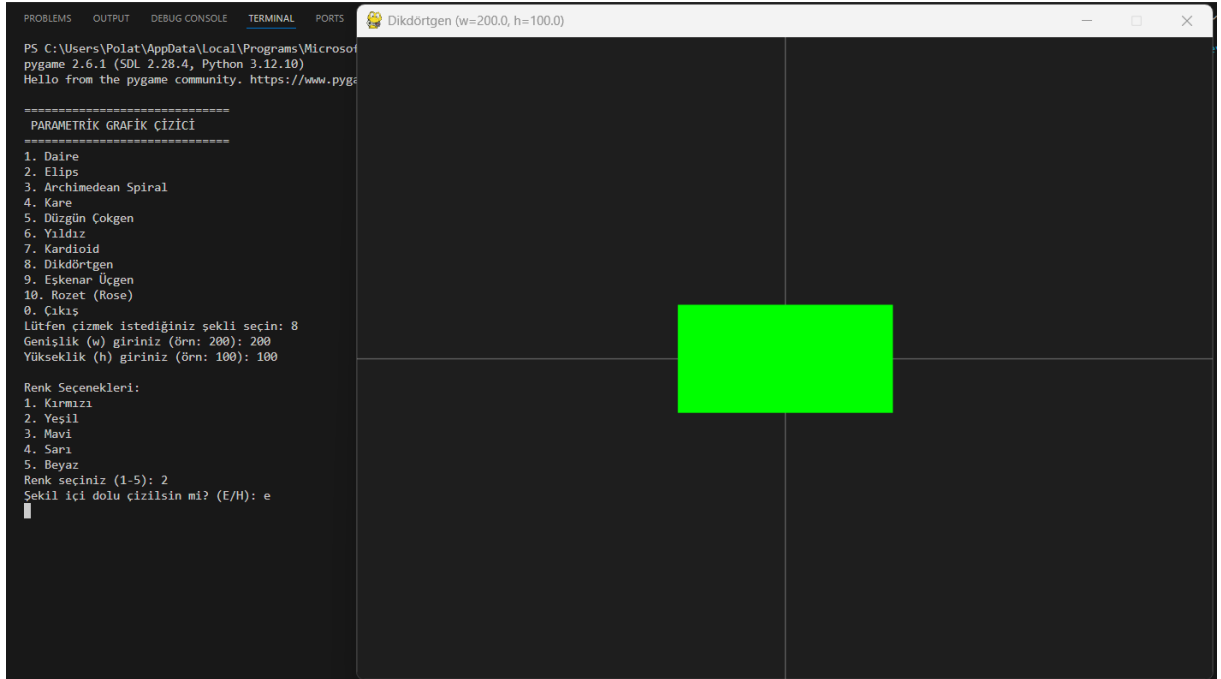
Şekil 5: Mavi renkli, içi dolu Çokgen çizimi



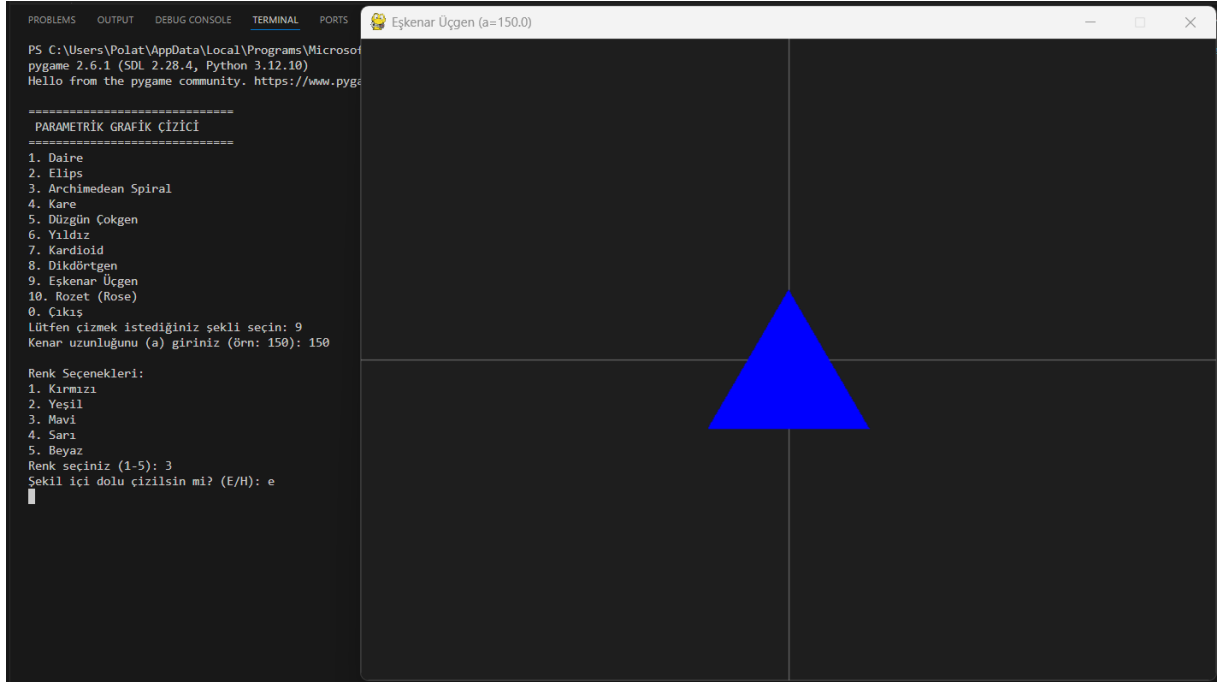
Şekil 6: Kırmızı renkli, içi dolu Yıldız çizimi



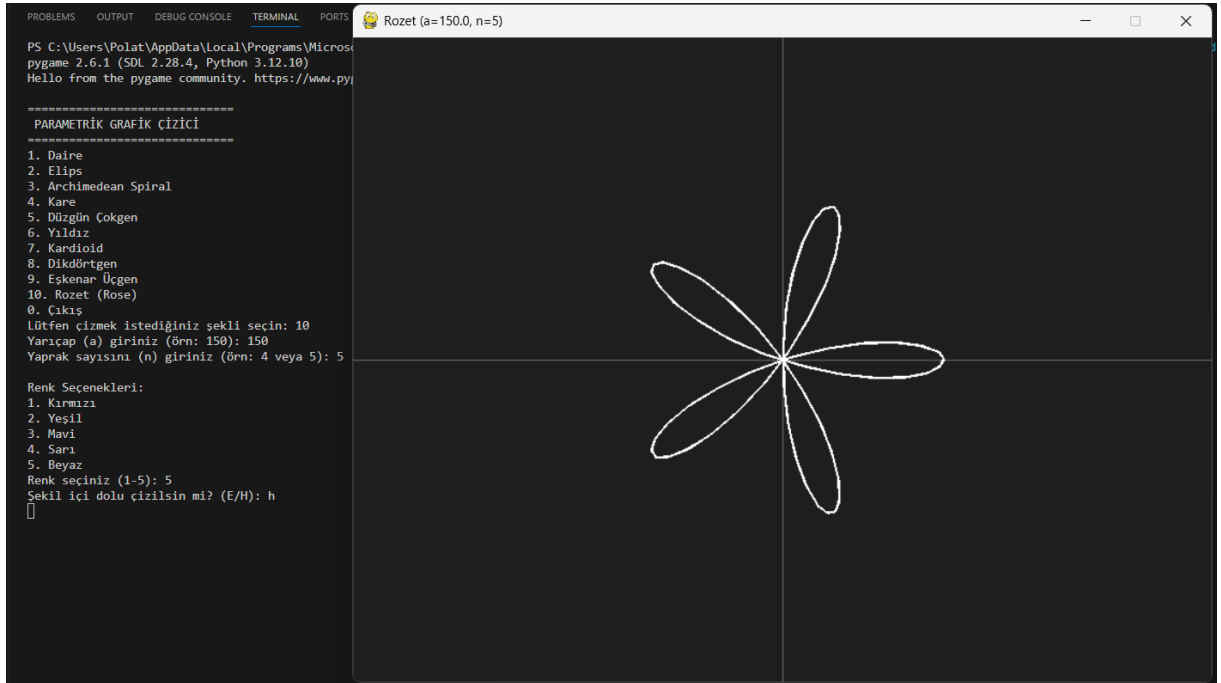
Şekil 7: Sarı renkli, içi dolu Kardioid çizimi



Şekil 8: Yeşil renkli, içi dolu Dikdörtgen çizimi



Şekil 9: Mavi renkli, içi dolu Eşkenar Üçgen çizimi



Şekil 10: Beyaz renkli, içi boş Rozet (Rose) çizimi

5. Karşılaşılan Optimizasyon Sorunları ve Çözümler

Proje geliştirme sürecinde, hem donanımsal mantık hem de yazılımsal algoritmalar açısından iki temel zorlukla karşılaşılmış ve başarıyla çözülmüştür:

- **Koordinat Sistemi Zıtlığı (Inversion Problem):** Piksel tabanlı bilgisayar ekranlarında orijin noktası (0,0) sol üst köşededir ve Y eksenini aşağı yönde artırır. Matematiksel Kartezyen sistemde ise Y eksenini yukarı yönlüdür. Bu zıtlık grafiğin ters çizilmesine neden olduğundan, aracı bir dönüşüm katmanı yazılarak Y ekseninin değeri ekran merkezinden çıkarılmış ($\text{Merkez_Y} - y$) ve sistemler arası köprü kurulmuştur.
- **Kendi Kendini Kesen Eğrilerde (Self-Intersecting) Dolgu Anomalisi:** Rozet, Yıldız ve Kardioid gibi çizgilerin kendi üzerinden geçtiği parametrik eğrilerde, grafik motorlarının standart çokgen dolgu algoritmaları (Sarma Kuralı / Winding Rule) nerenin "içerisi" nerenin "dışarı" olduğuna karar veremez. Bu durum görsel yırtılmalara yol açmıştır. Sisteme parametrik esneklik katılarak, kullanıcıya şekli "çizgisel" veya "dolgulu" çizme opsiyonu sunulmuş; böylece karmaşık yolların doğasına uygun şekilde pürüzsüzce render edilmesi sağlanmıştır.

6. Kaynakça

- Python Software Foundation. (2026). Python 3.12 Documentation: Math Libraries.
- Pygame Community. (2026). Pygame Draw Module Official Documentation.
- Iğdır Üniversitesi Grafik Programlama Dersi, Ödev-1 Teorik Altyapı ve Parametrik Temsiller Dokümanı.